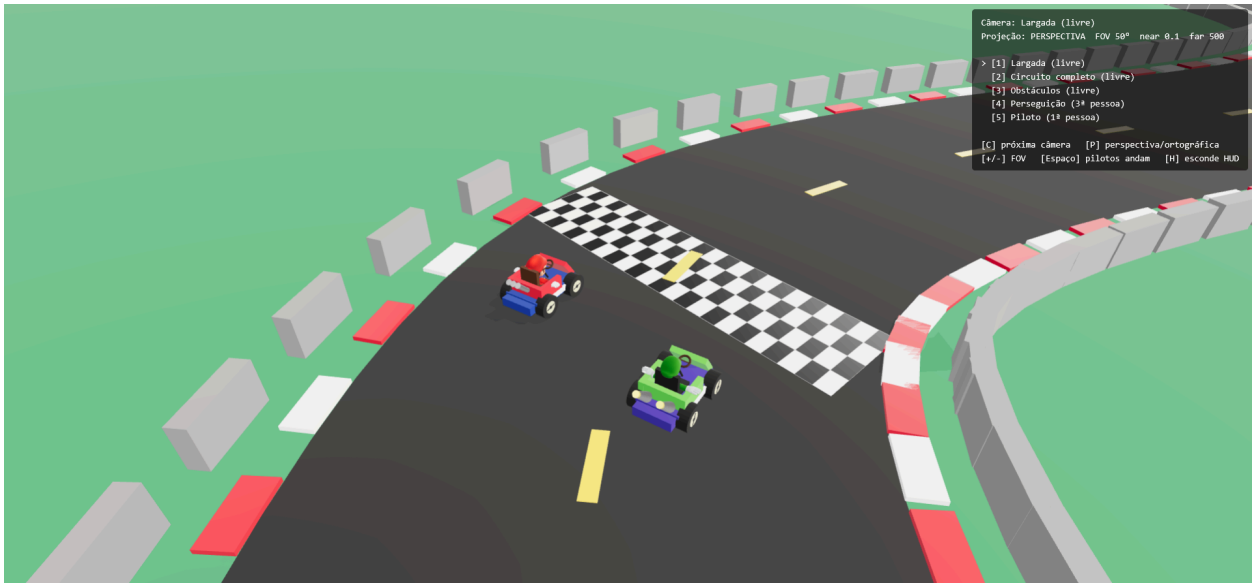
Relatório da disciplina de Computação Gráfica - Mario Kart 3D (Entrega 3)

O atual relatório apresenta o que foi alterado da Entrega 2 para a Entrega 3. Nesta versão, o projeto ganhou um sistema de câmeras, o uso explícito da projeção perspectiva e um conjunto de fontes de luz. Tudo foi feito sobre a cena da entrega anterior, sem refazer os modelos.



[Figura 1: vista de largada (tecla 1) com a nova iluminação, mostrando Mario e Luigi no grid e a luz azul sobre a faixa de largada.]

1. A cena 3D

A cena continua montada em "src/main.js" e reúne tudo o que foi feito antes: a pista ("Track"), os dois pilotos ("Racer", que junta o kart procedural e o personagem em ".gltf") e os objetos de cenário (cones, barris, pilhas de pneus, manchas de óleo e caixas de item). Esses módulos não foram alterados nesta entrega, só passaram a ser usados por quem é novo: o "CameraManager" e o "LightController".

O que foi alterado na cena:

- O código de câmera e de luz saiu do "main.js". Antes havia uma câmera única e uma DirectionalLight escritas diretamente no arquivo. Agora cada assunto tem a sua classe e o "main.js" só chama as duas.
- Foi criado um "modo demo", ligado e desligado pela tecla **Espaço**. Na versão anterior, nada saía do lugar, o que tornava as câmeras de perseguição e de primeira pessoa inúteis. No modo demo cada piloto guarda a sua posição na curva ("userData.u") e o seu deslocamento lateral ("userData.side"). A cada frame, "driveRacer" avança "u" a 14 unidades por segundo, pega o

ponto e a tangente da curva ("getPointAt" e "getTangentAt") e reposiciona e gira o piloto. É o mesmo conceito usado no posicionamento dos obstáculos, só que repetido a cada quadro.
- A velocidade do kart passou a ser a do demo, então as rodas giram por "ω = v / r" como antes, mas agora acompanhando o movimento de verdade (com 14 u/s e raio 0,32 dá cerca de 44 rad/s). Na largada elas continuam girando no lugar com "idleWheelSpeed".
- A caixas de item continuam flutuando com "Math.sin" e girando, agora vistas de várias posições de câmera.

Esta versão compõe um circuito com asfalto, zebras (faixas vermelho/branco nas laterais da pista), muros, obstáculos e dois karts, que pode ser visualizado numa câmera aérea, por trás dos personagens ou por dentro do kart (primeira pessoa).

2. Múltiplas posições de câmera

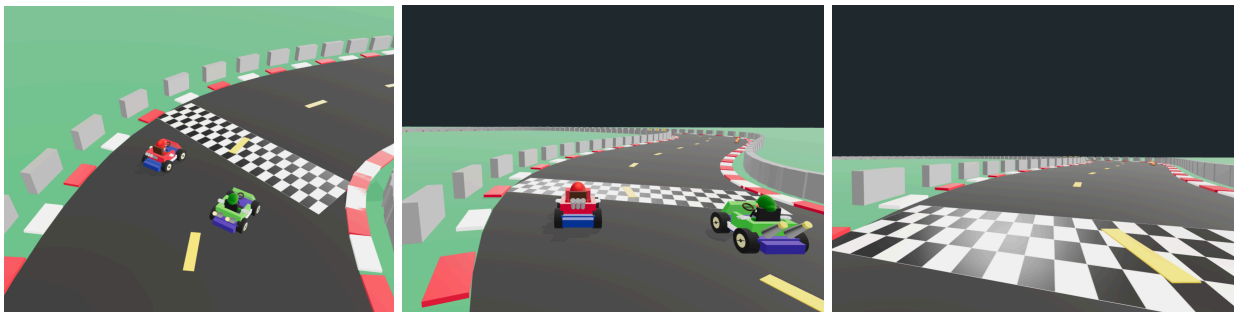
Todo o código está em "src/cameras/CameraManager.js". A classe guarda uma lista de visões, cada uma com a sua própria "PerspectiveCamera":

Tecla	Visão	FOV	Comportamento
1	Largada (livre)	50°	OrbitControls, começando atrás e acima do grid
2	Circuito completo (livre)	50°	OrbitControls, vista de cima a 245 unidades de altura
3	Obstáculos (livre)	50°	OrbitControls, posicionada no trecho dos cones
4	Perseguição (3ª pessoa)	65°	Fica 8 unidades atrás e 3,4 acima do Mario e olha para a frente dele
5	Piloto (1ª pessoa)	80°	Presa ao kart do Mario, na altura dos olhos

As visões 1, 2 e 3 são as mesmas posições da entrega anterior, mas agora reaproveitam uma câmera e mudam sua posição e o alvo do "OrbitControls". As visões 4 e 5 são novas; funcionam pois o "main.js" avisa ao gerenciador quem deve ser seguido, com "setFollowTarget(marioRacer)".

A troca acontece pelas teclas 1 a 5 ou pela tecla **C**, que passa para a próxima. Um HUD no canto superior direito mostra a câmera ativa, o tipo de projeção e o FOV, e pode ser escondido com **H**.

Para verificar que a troca altera o ponto de vista de fato, basta ligar o modo demo e alternar: na 1 os dois karts aparecem do alto e parados, na 4 o Mario aparece de costas com a pista inteira à frente e na 5 a pista é vista como o piloto a enxerga.



[Figura 2: as três visões lado a lado, na ordem 1, 4 e 5.]

Obs: Na perseguição, a posição da câmera é suavizada com "lerp" usando o fator "1 - e^(-6·delta)", que mantém a suavização igual em qualquer taxa de quadros e evita o tremor nas curvas; o "lookAt" não é suavizado. Já o "OrbitControls" é desligado nas visões 4 e 5 ("controls.enabled = false"), pois ele entraria em conflito com o "lookAt" manual a cada frame.

3. Projeção perspectiva

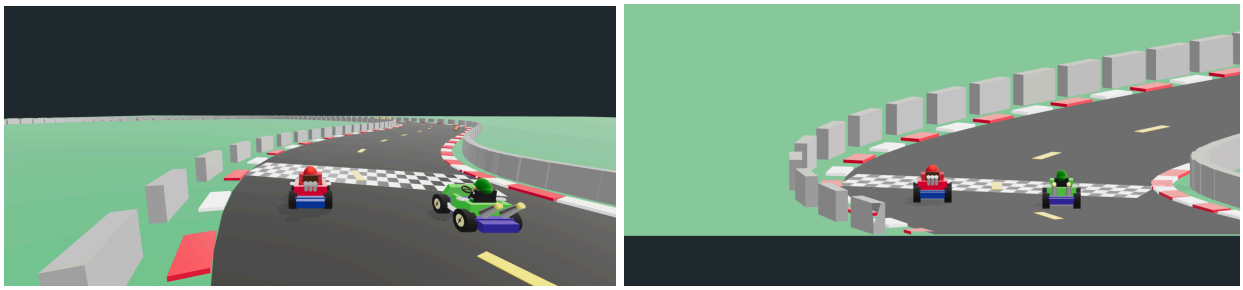
Todas as câmeras de visualização são "THREE.PerspectiveCamera", com "near = 0.1" e "far = 500" (o valor de "far" é o que permite ver o circuito inteiro pela tecla 2).

A matriz é definida em "CameraManager._makeCamera()", na linha "new THREE.PerspectiveCamera(fov, aspect, near, far)". O construtor chama "updateProjectionMatrix()", que monta a matriz 4x4 de projeção em "camera.projectionMatrix". Ela é refeita sempre que um parâmetro muda: em "resize()", quando o aspecto da janela muda, e em "changeFov()", quando o FOV é alterado pelas teclas **+** e **-**.

A matriz é aplicada na chamada "renderer.render(scene, cameraManager.camera)" dentro do loop de animação do "main.js". O Three.js envia a "projectionMatrix" e a matriz de visão para a GPU, e o vertex shader calcula "gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0)". A divisão por "w", feita depois pela própria GPU, é o que faz objetos distantes ficarem menores.

Para demonstrar o efeito, usamos três recursos:

- Na visão 4 com o demo ligado, cones e caixas de item aparecem pequenos ao longe e vão crescendo conforme o kart se aproxima, e as bordas da pista convergem para o horizonte.
- A tecla **P** troca para uma "OrthographicCamera", só para comparação. Ela copia a posição e a rotação da câmera ativa e usa a altura visível "h = d · tan(fov/2)", sendo "d" a distância até o ponto de referência. Assim o objeto de referência tem o mesmo tamanho nas duas projeções e só a perspectiva muda. Na ortográfica o Mario e o Luigi, a distâncias diferentes da câmera, aparecem do mesmo tamanho e a pista deixa de convergir. Na visão do piloto esse botão não faz nada, porque o chão fica rente ao horizonte e a comparação não é útil.
- As teclas **+** e **-** mexem no FOV (de 20° a 120°). Com FOV grande a cena fica com um aspecto grande angular, e a distorção nas bordas aumenta; o FOV pequeno tem um aspecto de zoom, e a cena fica achatada.



[Figura 3: o mesmo trecho na vista 4 em perspectiva e em ortográfica (tecla P).]

4. Múltiplas fontes de luz

As luzes ficam em "src/lighting/LightController.js", que cria as fontes e guarda todas em "this.lights". São três:

Fonte	Tipo	Cor e intensidade	Papel
ambient	AmbientLight	branca, 0,3	luz de base, para nada ficar totalmente preto na sombra
sun	DirectionalLight	tom quente (0xfff5e6), 1,2	luz principal do dia, em (50, 100, 50), com sombras em mapa de 2048
trackSpot	SpotLight	azul (0x00aaff), 2,0	holofote a 20 unidades de altura em (0, 20, 0), com cone de 45° e penumbra 0,5

O ponto (0, 0, 0) é justamente o ponto "u = 0" da curva, ou seja, a linha de largada. Por isso o holofote ilumina o grid onde Mario e Luigi esperam o start, como uma luz de largada de estádio, e o azul o diferencia do sol amarelado. A luz direcional é o sol da pista e dá volume aos karts, muros e zebras pelas sombras, por ora só na região da largada (ver seção 6).

Para identificar a contribuição de cada fonte, a classe tem o método "toggleLight(nome, ligado)", que liga e desliga qualquer luz pelo nome ("ambient", "sun" ou "trackSpot"), e "setIntensity", que ajusta a intensidade. Nesta entrega, porém, a contribuição de cada fonte é verificada chamando "toggleLight" no código. Com só a ambiente a cena fica chapada e sem sombras; só com o sol aparecem o volume e as sombras; só com o holofote vê-se o círculo azul no grid.

Obs: a cena tem também o mapa de ambiente ("RoomEnvironment" processado com "PMREMGenerator"), que veio da entrega anterior para o Luigi não ficar escuro demais. Ele não é uma fonte de luz do "LightController" e continua ligado quando se desliga as três, então a cena nunca fica totalmente escura.

5. Principais dificuldades

- No começo o "OrbitControls" continuava ativo nas vistas de perseguição e de piloto e puxava a câmera de volta. Resolvemos habilitando o controle só nas vistas livres.

- A primeira versão da ortográfica deixava os objetos com tamanhos totalmente diferentes dos da perspectiva, o que tirava o sentido da comparação. A solução foi calcular a altura visível a partir da distância e do FOV, como mostrado acima.

- Com as luzes antigas trocadas pelas três novas, foi preciso reequilibrar as intensidades, pois o sol sozinho deixava as sombras muito escuras e a ambiente alta demais apagava o volume dos objetos.

6. O que ainda falta implementar

- Ligar os atalhos de luz (teclas 8, 9 e 0): chamar "setupKeyboardShortcuts()" no "main.js" e trocar "lights.spot" por "lights.trackSpot" na tecla 0.
- Cobrir o circuito inteiro com as sombras do sol. A câmera de sombra da luz direcional ainda está com o tamanho padrão do Three.js e o bloco antigo, que cobria 230 unidades, ficou comentado no "main.js".
- Faróis nos karts. O "LightController" já tem o método "update(kartPosition)" preparado para uma luz móvel, mas ela ainda não foi criada.
- Movimento de verdade. O modo demo só leva os pilotos por uma curva fixa; falta controle por teclado, aceleração, colisão e a lógica de corrida.
- Resolver as mãos dos personagens que atravessam a carroceria, pendente desde a entrega anterior.
- Som, itens e adversários, como já descrito na Entrega 2.